



## Free Hosting Guide

Step-by-Step: Launch Your App at Zero Cost

Olusegun Abayomi Stephen · 2024

100% Free

No Credit Card

Production Ready

Custom Domain

Render

Railway

Vercel

Cloudflare

Supabase

Backblaze  
B2

### What this guide covers

This guide shows you exactly how to host the complete MyLover app — backend API, React frontend, file uploads, and database — entirely free of charge using modern cloud platforms. You will be able to add a custom domain (e.g. mylover.app) and serve real users. When you start earning, upgrading to paid plans takes one click.

## Free Platform Comparison

Different parts of the app are hosted on different free services — this is the industry-standard "Jamstack" approach. Each service is best-in-class for its purpose.

| Service          | What It Hosts                            | Free Limits                                             | Upgrade Cost        |
|------------------|------------------------------------------|---------------------------------------------------------|---------------------|
| Render.com       | Node.js backend (API + Socket.io)        | 750 hrs/month, 512MB RAM, spins down after 15 mins idle | \$7/month always-on |
| Railway.app      | Node.js backend (alternative to Render)  | \$5 credit/month free — ~500 hrs                        | \$5+/month for more |
| Vercel           | React frontend (static build)            | 100GB bandwidth, unlimited deploys                      | \$20/month Pro      |
| Cloudflare Pages | React frontend (alternative to Vercel)   | 1GB limited bandwidth, 500 builds/month                 | \$5/month Pro       |
| Supabase         | PostgreSQL database (production upgrade) | 500MB DB, 1GB storage, 50k MAU                          | \$25/month Pro      |
| Backblaze B2     | User photo storage (upgrade)             | 10GB free, \$0.006/GB/month after                       | Pay as you go       |

|                      |                                  |                        |                             |
|----------------------|----------------------------------|------------------------|-----------------------------|
| Cloudflare R2        | User photo storage (alternative) | 10GB/month free egress | Pay as you go               |
| FreeDNS / Cloudflare | Free custom domain or DNS        | Free forever           | \$0 to \$15/year for domain |

# Option A — Render (Backend) + Vercel (Frontend)

**Recommended for MyLover.** Render handles your Node.js server including Socket.io. Vercel serves the React build with a global CDN. Both have free tiers that handle hundreds of users with no credit card required.

## PART 1 — Deploy the Backend to Render

### Step 1: Prepare your code repository

- Create a free account at **github.com** if you do not have one
- Create a new repository (e.g. mylover-backend), set it to Private
- Push your entire mylover/ folder to GitHub:

```
git init
git add .
git commit -m "Initial MyLover backend"
git remote add origin https://github.com/YOUR_USERNAME/mylover-backend.git
git push -u origin main
```

### Step 2: Sign up on Render

- Go to **render.com** and click "Get Started for Free"
- Sign up with your GitHub account (recommended — allows auto-deploy)
- You do not need to enter payment details for the free plan

### Step 3: Create a new Web Service on Render

- From the Render dashboard, click **New** → **Web Service**
- Connect your GitHub account and select the mylover-backend repository
- Configure the service settings as follows:

```
Name: mylover-backend Region: Ohio (US East) or Frankfurt (EU) Branch: main
Root Dir: server Runtime: Node Build Cmd: npm install Start Cmd: node
index.js Instance: Free
```

- Click **Advanced** → Add Environment Variables (see Step 4)

### Step 4: Set environment variables on Render

- In the Render service dashboard, go to **Environment** tab
- Add each variable from server/.env.example:

```
NODE_ENV = production PORT = 10000 JWT_SECRET = (generate a long random
string – 64+ characters) CLIENT_URL = https://your-app.vercel.app (fill in
after Step 8)
```

Generate a secure JWT\_SECRET with: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`

- Click **Save Changes** — Render will auto-deploy

### Step 5: Verify backend is live

- After deploy completes (~3 minutes), Render gives you a URL like:  
`https://mylover-backend.onrender.com`
- Open that URL + `/api/health` in your browser

Expected response: `{"status":"ok", "app":"MyLover", "version":"1.0.0"}`

- Also check: `https://mylover-backend.onrender.com/api/health`

On the free plan Render spins down after 15 min of inactivity. The first request after sleep takes ~30 seconds. Upgrade to the \$7/month Starter plan for always-on performance.

## PART 2 — Deploy the React Frontend to Vercel

### Step 6: Build the React app for production

- In your project, update **client/src/Utils/api.js**:

```
// Change the BASE constant at the top of api.js: const BASE =  
process.env.REACT_APP_API_URL || ''; // This allows you to set the API URL  
via an environment variable in Vercel
```

- Create a Vercel config file in the **client/** folder:

```
// client/vercel.json { "rewrites": [{ "source": "/(.*)", "destination": "/"  
}] }
```

- Commit and push these changes to GitHub

### Step 7: Sign up on Vercel

- Go to **vercel.com** and click "Start Deploying"
- Sign up with your GitHub account
- No payment details needed for the free Hobby plan

### Step 8: Import and deploy the frontend

- From the Vercel dashboard, click **Add New** → **Project**
- Import your mylover-backend GitHub repository
- Configure project settings:

```
Framework Preset: Create React App Root Directory: client Build Command: npm  
run build Output Directory: build Install Command: npm install  
--legacy-peer-deps
```

- Under **Environment Variables**, add:

```
REACT_APP_API_URL = https://mylover-backend.onrender.com (your Render  
backend URL from Step 5)
```

- Click **Deploy** — Vercel builds and deploys in ~2 minutes
- You get a URL like: https://mylover-XXXX.vercel.app

### Step 9: Update CORS on the backend

- Go back to your Render service dashboard → Environment tab
- Update the CLIENT\_URL variable to your Vercel URL:

```
CLIENT_URL = https://mylover-XXXX.vercel.app
```

- Click **Save Changes** — Render auto-redeploys

Socket.io uses the CLIENT\_URL for CORS. If you skip this, real-time chat will be blocked.



## PART 3 — Add a Free Custom Domain

### Step 10: Get a free domain or use a subdomain

- **Option A — Free subdomain (no cost ever):** Vercel gives you `yourapp.vercel.app` and Render gives you `yourapp.onrender.com`. Use these as-is.
- **Option B — Buy a cheap domain:** `.app` domains cost ~\$14/year at Namecheap or Google Domains. Search for `mylover.app`, `findlove.app`, etc.
- **Option C — Freenom:** Offers free `.tk` and `.ml` domains (go to [freenom.com](https://freenom.com)). Less trusted by email clients but works for testing.

### Step 11: Connect domain to Vercel (frontend)

- In Vercel project → **Settings** → **Domains**
- Click "Add Domain" and enter your domain (e.g. `mylover.app`)
- Vercel shows you DNS records to add
- At your domain registrar, add the CNAME record Vercel provides:

Type: CNAME Name: `www` (or `@`) Value: `cname.vercel-dns.com`

- DNS propagation takes 5-30 minutes. Vercel auto-provisions a free SSL certificate via Let's Encrypt.

### Step 12: Connect domain to Render (backend API)

- In Render service → **Settings** → **Custom Domains**
- Add a subdomain like `api.mylover.app`
- Add the CNAME record at your registrar:

Type: CNAME Name: `api` Value: `mylover-backend.onrender.com`

- Update Render environment: `CLIENT_URL` = `https://mylover.app`
- Update Vercel environment: `REACT_APP_API_URL` = `https://api.mylover.app`
- Redeploy both services. You now have a fully branded app!

## Option B — Railway (Backend Alternative)

---

Railway is simpler to set up than Render and gives \$5 of free credit per month. It does NOT spin down inactive services. Use Railway for the backend if you prefer always-on performance on the free tier.

|             |                                                                                                                                                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sign up     | Go to <a href="https://railway.app">railway.app</a> and sign up with GitHub. The Starter plan gives \$5 credit/month — enough for ~500 hours of a small Node.js app.                                                                              |
| New Project | Click "New Project" → "Deploy from GitHub repo". Select your repository. Railway auto-detects Node.js.                                                                                                                                            |
| Configure   | Set the Start Command: <code>node server/index.js</code><br>(or if root dir is <code>server/</code> : <code>node index.js</code> ). Railway auto-runs <code>npm install</code> .                                                                  |
| Variables   | Go to Variables tab and add:<br>PORT = 3000 (Railway uses this automatically)<br>NODE_ENV = production<br>JWT_SECRET = (your long random key)<br>CLIENT_URL = <a href="https://your-vercel-app.vercel.app">https://your-vercel-app.vercel.app</a> |
| Domain      | Go to Settings → Domains → Generate Domain. You get a free <a href="https://railway.app">railway.app</a> subdomain.                                                                                                                               |
| Frontend    | Deploy the React frontend to Vercel exactly as in Option A Steps 6-9, pointing <code>REACT_APP_API_URL</code> to your Railway domain.                                                                                                             |



## Database — Production Upgrade Path

---

The app ships with **SQLite via sql.js** which is perfectly fine for development and early-stage production (up to a few hundred concurrent users). When you start scaling, migrate to PostgreSQL. The query API is identical so the code change is minimal.

### Free PostgreSQL with Supabase

|   |                                                                                                                                |
|---|--------------------------------------------------------------------------------------------------------------------------------|
| 1 | Go to <a href="https://supabase.com">supabase.com</a> and create a free account                                                |
| 2 | Create a new project (choose a region close to your users — e.g. West Africa → EU West)                                        |
| 3 | Go to Project Settings → Database → Connection String (URI format)                                                             |
| 4 | Copy the connection string:<br><code>postgresql://postgres:PASSWORD@db.XXXXXX.supabase.co:5432/postgres</code>                 |
| 5 | Add to your Render/Railway environment variables: <code>DATABASE_URL</code> = (the connection string)                          |
| 6 | Install the pg driver in server/: <code>npm install pg</code>                                                                  |
| 7 | Replace the <code>db_sqlite_shim</code> import in <code>db.js</code> with a <code>pg Pool</code> — all queries remain the same |
| 8 | Free tier: 500MB database, 1GB file storage, 50,000 monthly active users                                                       |

## Photo Storage — Free Cloud Storage

---

By default, uploaded photos are saved to the server's local filesystem (`server/uploads/`). This works on Railway (persistent disk) but NOT on Render free tier (ephemeral filesystem — files are deleted on every redeploy). Use Cloudflare R2 or Backblaze B2 for persistent free storage.

### Free Photo Storage with Cloudflare R2

|   |                                                                                                  |
|---|--------------------------------------------------------------------------------------------------|
| 1 | Go to <a href="https://cloudflare.com">cloudflare.com</a> → sign up free                         |
| 2 | From the dashboard, go to R2 Object Storage → Create bucket (e.g. <code>mylover-uploads</code> ) |
| 3 | Go to R2 → Manage R2 API Tokens → Create Token with Object Read & Write permissions              |
| 4 | Note your: Account ID, Access Key ID, Secret Access Key, Bucket Name                             |

|   |                                                                                                     |
|---|-----------------------------------------------------------------------------------------------------|
| 5 | Install the S3-compatible SDK in server/: npm install @aws-sdk/client-s3                            |
| 6 | Update server/routes/upload.js to use S3 storage instead of multer disk storage                     |
| 7 | R2 free tier: 10GB storage, 1 million Class A operations/month, 10 million Class B operations/month |
| 8 | R2 has ZERO egress fees — you never pay for downloads (unlike AWS S3)                               |

# Complete Free Stack Summary

This is the recommended zero-cost production setup for MyLover. It can comfortably handle your first 1,000 users completely free.

| Component       | Service                              | URL                                         | Free Limit                                          |
|-----------------|--------------------------------------|---------------------------------------------|-----------------------------------------------------|
| React Frontend  | Vercel                               | mylover.vercel.app<br>or your custom domain | Unlimited bandwidth<br>100GB/month                  |
| Node.js Backend | Render.com                           | mylover-api.onrender.com                    | 750 hrs/month<br>(spin-down after 15 min)           |
| Database        | SQLite file<br>(on Render disk)      | Stored on server                            | Included with server<br>(upgrade to Supabase later) |
| Photo Storage   | Render disk<br>(or Cloudflare R2)    | Served via /uploads                         | 1GB disk on Render free<br>(10GB R2 for persistent) |
| DNS / CDN       | Cloudflare                           | Your domain                                 | Free forever                                        |
| SSL Certificate | Let's Encrypt<br>(via Vercel/Render) | Auto-renewed                                | Free forever                                        |
| AI Features     | Anthropic API<br>(via Claude.ai)     | api.anthropic.com                           | Included in this build                              |
| STUN (WebRTC)   | Google STUN                          | stun.l.google.com                           | Free, unlimited                                     |

# When You Start Earning — Upgrade Path

|                  |                                                                                                                    |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| First \$7/month  | <b>Render Starter plan</b><br>Eliminates the 15-minute spin-down. Your API responds instantly 24/7.                |
| First \$20/month | <b>Render Standard</b><br>1GB RAM — handles ~1,000 concurrent socket connections comfortably.                      |
| First \$25/month | <b>Supabase Pro</b><br>Migrate from SQLite to PostgreSQL. 8GB DB, no pause, daily backups.                         |
| First \$50/month | <b>Paystack integration</b><br>Accept real card payments in Nigeria (1.5% + ₦100 per transaction). No monthly fee. |
| Growing          | <b>DigitalOcean Droplet</b><br>\$6/month for a VPS. Full control, better performance than managed PaaS.            |

**Important:** The app is production-code quality right now. The only thing separating you from live users is running "npm install" and pushing to GitHub. Start free, grow with your revenue, and never over-invest before you have users.